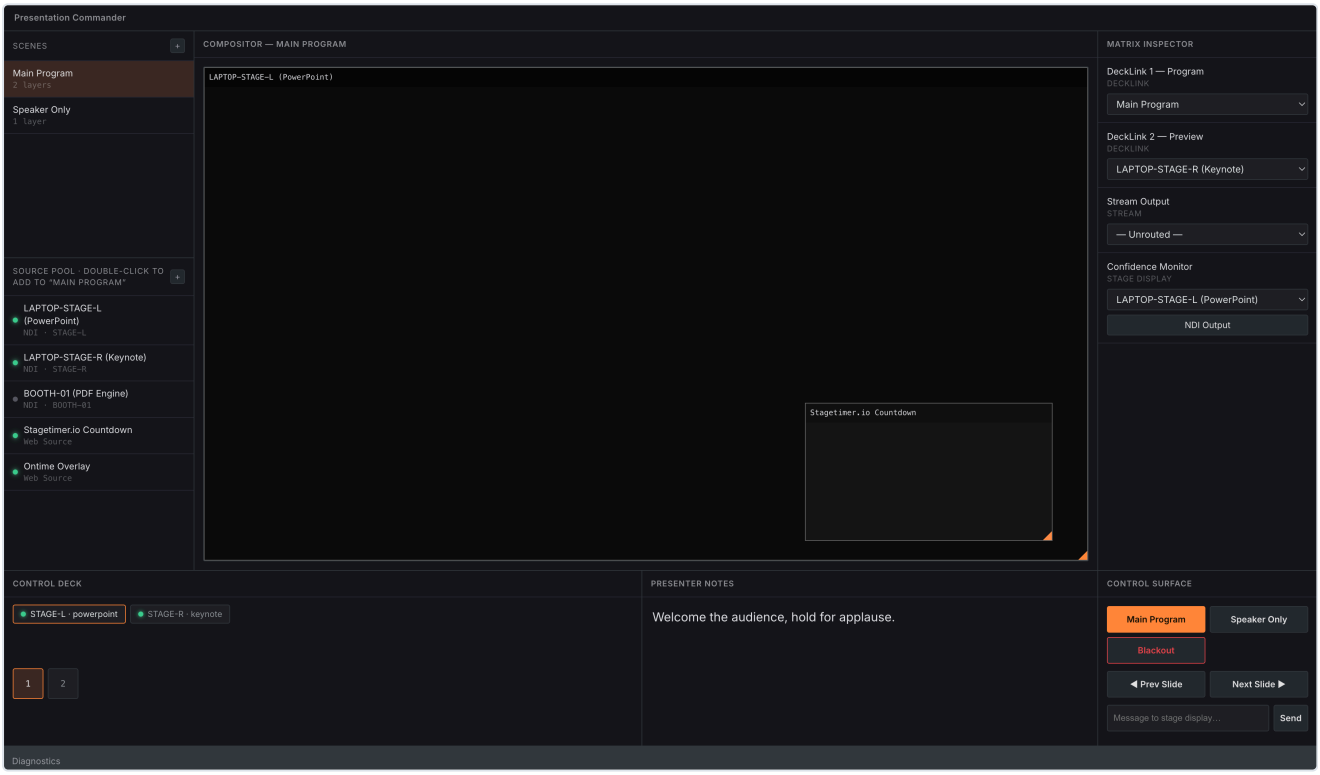
Presentation Commander Server user guide

Presentation Commander Server is the master control app for a live event: an NDI video matrix router, a layered scene compositor and a presenter-notes hub for the laptops running the client app.

This guide is for the operator running the show. The README covers installing it and getting past the unsigned-build warnings. This is what to do once it's open, and what to be careful with.



The main window: scene compositor, source pool, matrix inspector, control deck and control surface.

What is real, and what is not

This matters more here than in most apps, because the interface shows you a broadcast router and some of it is a model rather than a machine.

Feature	State
NDI **discovery** (finding senders on the network)	**Real** — mDNS, same mechanism NDI uses
NDI **receive** (live video in the compositor)	**Real** — official Vizrt NDI SDK, decoded frames
NDI **send** (the confidence monitor output)	**Real** — a genuine NDI sender other devices can subscribe to
Web sources, scenes, layering, routing	**Real** — within the app
Client Hub: notes, slide position, next/previous	**Real** — when a client is connected
DeckLink and other physical broadcast I/O	**Not implemented.** No signal leaves a card.

> **Routing to a "DeckLink" output changes the picture in this app and nothing else.** There's no capture-card driver. It's out of scope, because I've no hardware to test it against. The output exists in the model so the routing logic is right, but it doesn't put video on an SDI connector. The one output that actually reaches other equipment is the NDI confidence-monitor sender.

This is an AI-assisted codebase, directed and reviewed by a human author. **Check it over before you rely on it for a show**, as the README says, and I'd stand by that. It hasn't been proven on a live show.

First launch: clear out the demo data

The app starts with **seed data baked in**, and it looks like a configured rig:

- three NDI sources (`LAPTOP-STAGE-L (PowerPoint)`, `LAPTOP-STAGE-R (Keynote)`, `BOOTH-01 (PDF Engine)`)
- two web sources (Stagetimer.io, Ontime)
- two scenes (`Main Program`, `Speaker Only`)
- two client nodes (`STAGE-L`, `STAGE-R`) with presenter notes
- outputs pre-routed

None of it is real. `LAPTOP-STAGE-L` isn't on your network. Delete the sources and scenes you don't want before you build anything, or sooner or later you'll route a black frame to a screen because the thing you routed never existed.

The two demo *clients* can't be deleted from the UI. They only go away when the app is restarted, which also throws away your own work (Nothing is saved).

Nothing is saved

All state lives in memory. There's no project file, no autosave and no recent-documents list.

Quitting the app throws away every source you added, every scene you built, and every route you set. The next launch comes back to the demo data in First launch: clear out the demo data.

In practice, on a show day:

- **Don't quit the app between rehearsal and doors.** Building the scene stack is the work, and you can't get it back.
- A crash costs you the same. There's no recovery.
- Plan for a rebuild: write your source list and scene layout down somewhere outside the app, because the app can't tell you what it had.
- There's no way to hand a configuration to a second machine, or to prepare one offline.

This is the biggest operational limitation of the current build.

Sources

There are three kinds, all added from the **Source Pool** panel.

NDI: the thing to know is that a source **picked from network discovery** carries a host and port, and that's what makes live preview work. A source added by **typing a machine name** has no port, will show `connected: false`, and **will never show video** in the compositor. If a layer stays a grey box, this is almost always why. Use the discovery picker.

Discovery is mDNS. It finds senders on the local link and doesn't cross subnets, so a sender on a different VLAN will never appear. That's how the network works, not a fault in the app.

Web: any URL, rendered in the compositor. Tick **transparent** for overlays that carry alpha (Stagetimer, Ontime, lower thirds), and leave it off for a full-frame page.

Notes: renders a connected Client Node's *current slide's* presenter note as a text layer. This is what makes a proper confidence monitor: composite a Notes source over a video source and send the result out as NDI (The confidence monitor).

Deleting a source is destructive and immediate

Removing a source **clears every output routed to it** and **strips it out of every scene** that used it. There's no confirmation and no undo. A source deleted while on air blacks out its outputs.

Scenes

A scene is a stack of layers. **The bottom of the list renders first, and the last layer is on top.** Drag to reposition, drag a corner to resize, and toggle visibility per layer.

Geometry is stored as **percentages of the canvas**, not pixels, so a scene keeps its layout at any output resolution.

A layer backed by a real network NDI source shows **live decoded video**, not a placeholder. That's the quickest way to check a source is actually arriving before you route it anywhere.

Deleting a scene clears any output routed to it, the same as with sources.

Routing

The **Matrix Inspector** routes each output to either a single source **or** a whole composited scene. There are four outputs and **the list is fixed**: you can't add, remove or rename one.

Output	Kind	Reaches real equipment?
DeckLink 1 — Program	`decklink`	**No** (What is real, and what is not)
DeckLink 2 — Preview	`decklink`	**No**
Stream Output	`stream`	**No**
Confidence Monitor	`stage-display`	Via the NDI sender (The confidence monitor)

Blackout is just "route to nothing". To undo it, route the output back. There's no separate blackout state that clears itself.

The confidence monitor

This is the one path that produces a signal other equipment can receive.

Build a scene containing a video source plus a **Notes** source, then start the NDI output. The composited frame (video with the presenter's live current-slide note over it) goes out as a proper NDI sender, so a stage monitor, a decoder or another NDI-capable device can subscribe to it. The presenter sees a broadcast picture rather than a text box in the operator's window.

Two behaviours worth knowing:

- The sender **repeats the last frame every second** as a keep-alive, so a static composite doesn't go stale for receivers that expect a steady feed. That means a frozen picture at the far end isn't proof the sender's died.

- The output control is a **toggle**. Pressing it while it's running stops the sender and ignores the name field.

Client Nodes and presenter notes

A Client Node is the client app running on a presentation laptop. It connects to this server's hub on port **9800**, registers, and streams its slide position and notes.

When one connects it **turns up as a routable source on its own**, named `"<name> (<app>)"`. You don't add it, and there's no point removing it (the next registration recreates it).

The **Control Deck** shows live notes and slide position per client. The **Control Surface** gives you scene recall, blackout, next/previous slide and send-note-to-stage as buttons.

Client names must be unique

Registration matches on **name**. Two laptops registering as `STAGE-L` **collapse into one client**: the second overwrites the first's platform and app, and both sets of slide state land on the same entry. Give every machine its own name before the show.

Next/previous slide can succeed without advancing anything

If the target client is **connected**, the command is passed on to it and the deck moves.

If it's **not connected**, the server falls back to *simulating it locally*: it moves its own idea of the slide index to the next slide it holds a note for. The button lights, the number in the UI changes, and **nothing happens on the presentation laptop**. If there are no notes for that client at all, nothing whatsoever happens, and the control still reports success.

Before you trust a slide button, check the client shows as **online**. The same goes for the Stream Deck buttons, which go through the same path.

Anyone on the network can register as a client

The hub listens on all interfaces with **no authentication**: no password, no token, no allowlist, no TLS. Any device that can reach port 9800 can register itself as a Client Node and push presenter notes that you'll see, and that a Notes source will composite onto a stage monitor.

Run this on a locked-down production network. It isn't safe on venue or guest Wi-Fi.

Stream Deck / Companion

The Companion module drives the server over HTTP on port **9700**, through the same command path as the in-app Control Surface, so a button and a click can't do different things.

That port is bound to `127.0.0.1` only, on purpose. It runs commands with no authentication whatsoever, so opening it to the network should be the operator's own conscious decision (an SSH tunnel or an authenticating reverse proxy), not a default. If Companion runs on a separate Stream Deck machine, see the module's README for how to reach it.

`send-note` has **one slot, not a queue**, so each message replaces the last. There's no clear button; send an empty message.

Troubleshooting

Symptom	Cause
A layer shows a grey box, never video	The NDI source was added by typing a name rather than picked from discovery, so it has no port and cannot be received (Sources).
A sender on the network never appears in discovery	mDNS doesn't cross subnets. Different VLAN = invisible.
Routed an output, nothing appeared on the screen	If it's a DeckLink or Stream output, that is expected — no physical I/O exists (What is real, and what is not).
Everything vanished after a restart	Expected. Nothing is persisted (Nothing is saved).
Demo sources are back	Same cause — the app reseeded (First launch: clear out the demo data).
Two laptops show as one client	They registered with the same name (Client Nodes and presenter notes).
Slide button "works" but the deck doesn't move	The client is offline and the server simulated it locally (Client Nodes and presenter notes).
Slide button does nothing at all	Client offline *and* no notes held for it — the fallback has nothing to move through.
A client's notes are stale	Notes are replaced wholesale on each sync, and timestamps are stamped at receive time, so they show the last sync, not when a note was written.
A client is sending but the server ignores it	Malformed JSON is dropped silently, and `slide-state` before `register` is ignored. Nothing is logged to the operator.
Deleting a source blacked out an output	Expected and immediate — deletion clears routes and scene layers (Sources).
Confidence monitor frozen at the far end	Not necessarily dead — the sender repeats the last frame every second by design (The confidence monitor).
Companion can't reach the server	Port 9700 is loopback-only. It needs a tunnel from another machine (Stream Deck / Companion).
macOS says the app is damaged	Not the released build — those are signed and notarised. A self-built or pre-notarisation copy is quarantined. Use the current release; UNSIGNED.md shows how to check a copy is the notarised one.

Before a show — a short checklist

1. Launch the app and **delete the demo sources and scenes** (First launch: clear out the demo data).
2. Add sources **from the discovery picker**, not by typing names (Sources).
3. Check each NDI layer shows live video before routing it (Scenes).
4. Check every client shows **online** before relying on a slide button (Client Nodes and presenter notes).
5. Be clear what actually reaches equipment: the **NDI confidence-monitor output**, and nothing else (What is real, and what is not).
6. **Don't quit the app** until you're done for the day (Nothing is saved).
7. Make sure the network this is on can't be reached by anyone you wouldn't hand the show to (Client Nodes and presenter notes, Stream Deck / Companion).

See also

- API.md: the automation API, client protocol and domain model
- DEVELOPING.md: building it
- README: install, unsigned-build warnings, architecture diagram

---